
Data Structure and Programming, Spring 2025

Programming Assignment #2

TA: 金巧芸
b10901154@ntu.edu.tw

Outline

1. Problem Description
2. Implementation Requirements
 - a. 1D Increasing Version (40%)
 - b. General 1D Version (30%)
 - c. 2D City (30%)
3. Evaluation
4. Submission
5. Grading policy

Problem Description

A city planner wants to find the Largest Land for Building.

input:

- A $m \times n$ array (city map)
- Each element is either empty (1) or an occupied (0)

output: The largest rectangular empty area

Problem Description

Input:



```
[[1, 1, 0, 0, 1],  
 [1, 0, 1, 1, 0],  
 [0, 1, 1, 1, 0],  
 [0, 0, 1, 1, 1]]
```

output: 6

Implementation Requirements

1. 1D Increasing Version (40%)
2. General 1D Version (30%)
3. 2D City (30%)

1D Increasing Version(40%)

Simplify the problem and express the city in 1-dimension

Input:

- An array with length n , representing land heights
- Each element in the array is non-negative integer and the array is non-decreasing

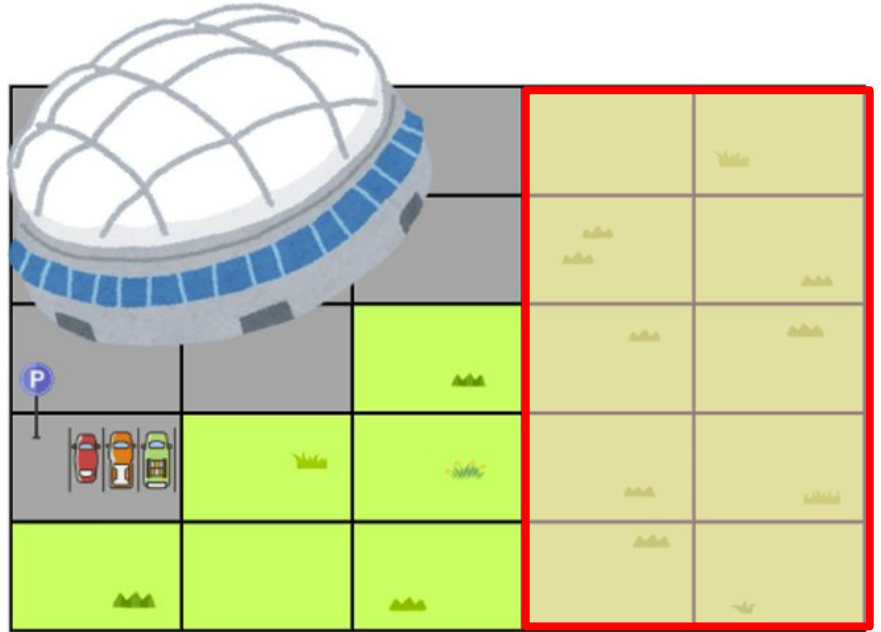
output: The largest rectangular area

1D Increasing Version(40%)

Input: [1, 2, 3, 5, 5]

output: 10

Return type: int
Run in $O(n)$



General 1D Version(30%)

Express the city in 1-dimension, but height not necessarily increasing.

Input:

- An array with length n , representing land heights
- Each element in the array is non-negative integer

output: The largest rectangular area

General 1D Version(30%)

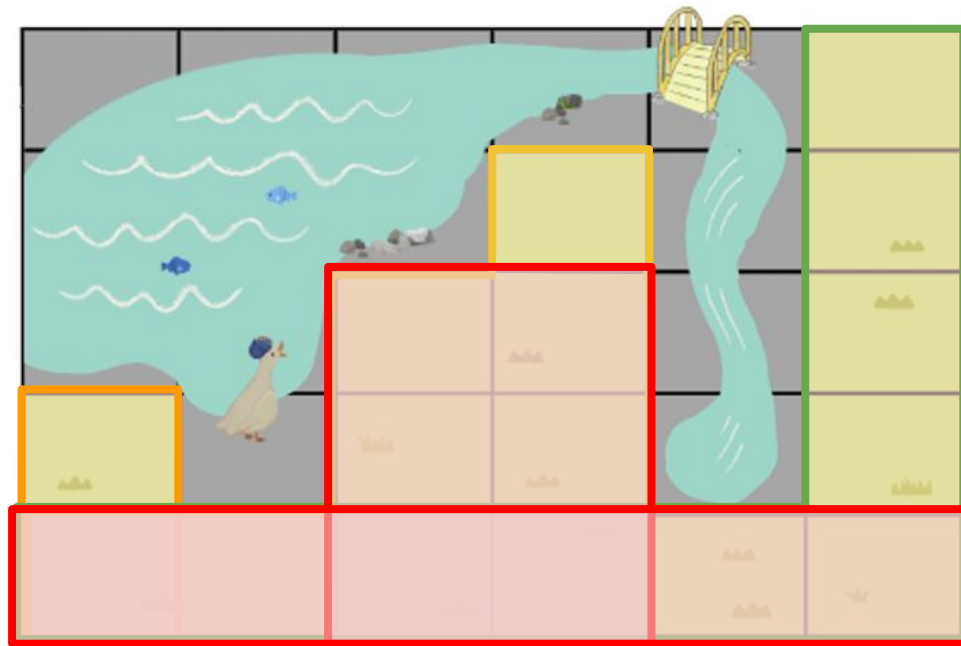
Return type: int
Run in $O(n)$

Input: [2, 1, 3, 4, 1, 5]

Output: 6

Hint:

- Multiple increasing segments
- Need to be processed with more efficient algorithms



2D City(30%)

Input:

- A $m \times n$ array (city map)
- Each element is either empty (1) or an occupied (0)

Output: The largest rectangular empty area

2D City(30%)

Input:



```
[[1, 1, 0, 0, 1],  
 [1, 0, 1, 1, 0],  
 [0, 1, 1, 1, 0],  
 [0, 0, 1, 1, 1]]
```

output: 6

2D City(30%)

Hint: Treat each row as a base and simplify it to another problem



```
[[1, 1, 0, 0, 1],  
 [1, 0, 1, 1, 0],  
 [0, 1, 1, 1, 0],  
 [0, 0, 1, 1, 1]]
```

Return type: int
Run in $O(mn)$

Note

- Do not modify the interface of the functions, but you may add your own functions.
- You may use NumPy, but other statistics libraries are not allowed.
- Pay attention to the time complexity of each function.
- There is a stack class in MaxPlace.py. You may decide whether to use it or not.

Evaluation

Use the commands "python3 testcase1.py", "python3 testcase2.py" and "python3 testcase3.py" to verify the correctness.

- ❖ testcase1.py → Ver_1D_increase
- ❖ testcase2.py → Ver_1D_general
- ❖ testcase3.py → Ver_2D

Submission

Please put maxPlace.py into a directory named studentID and compress the directory into studentID.zip to NTUCOOL. The homework is due on 04/17, at 23:59.

ex:

```
b10901154.zip/  
  b10901154/  
    maxPlace.py
```

Grading policy

- Correctness: 60%, Runtime: 40%
- Public test cases: 30%, Private test cases: 70%
- For each section:
 - 1D Increasing Version: 40%
 - General 1D Version: 30%
 - 2D City Grid: 30%